CL-SCA: A Contrastive Learning Approach for Profiled Side-Channel Analysis

Annyu Liu, An Wang, Shaofei Sun, Congming Wei, Yaoling Ding, Yongjuan Wang, and Liehuang Zhu *Senior Member, IEEE*

***Abstract*—Side-channel analysis (SCA) based on machine learning, particularly neural networks, has gained considerable attention in recent years. However, previous works predominantly focus on establishing connections between labels and related profiled traces. These approaches primarily capture label-related features and often overlook the connections between traces of the same label, resulting in the loss of some valuable information. Besides, the attack traces also contain valuable information that can be used in the training process to assist model learning. In this paper, we propose a profiled SCA approach based on contrastive learning named CL-SCA to address these issues. This approach extracts features by emphasizing the similarities among traces, thereby improving the effectiveness of key recovery while maintaining the advantages of the original SCA approach. Through experiments of different datasets from different platforms, we demonstrate that CL-SCA significantly outperforms other approaches. Moreover, by incorporating attack traces into the training process using our approach, we can further enhance its performance. This extension can improve the effectiveness of key recovery, which is fully verified through experiments on different datasets.**

***Index Terms*—Side-channel analysis, Contrastive learning, Neural networks, Profiled analysis.**

I. Introduction

THE widespread use of Internet of Things (IoT) technology has raised significant concerns about the hardware security of smart devices. Side-channel analysis (SCA) is one of the most well-known threats to hardware security, which has been extensively studied over the past three decades. SCA can recover the secret key by exploiting the unintentional physical information during the execution of cryptographic algorithms, such as power consumption, electromagnetic emission, and time deviation.

Traditional SCA typically involves using specialized equipment to gain physical access to a target system in order to capture physical information, which can be considered as local SCA. Recently, remote SCA is becoming increasingly popular. Malicious attackers can capture physical information without requiring proximity to the target system, such as fabric being shared among multi-tenant in the cloud services or the FPGA being part of a complex system on chip [1]–[4]. The shift from local to remote attack scenarios makes SCA more practical and more threatening.

Profiled side-channel analysis is regarded as one of the most promising techniques of SCA [5], [6]. It assumes that the attacker has a profiled device identical to the target device, and can construct the template of profiled device's leakage traces to recover the key. One of the traditional methods employed in profiled SCA is template attack [7]. However, it is important to note that this technique is primarily suitable for handling low-dimensional data and heavily relies on preprocessing methods to reduce the dimensionality of the traces.

In 2011, Hospodar *et al.* observed that profiled SCA can be viewed as a classification problem [8]. This insight paved the way for leveraging machine learning techniques to address the limitations of template attacks [9], [10], which often struggle with high-dimensional data. Among these techniques, neural networks have been extensively studied for their ability to automatically extract valuable features from leakage traces [11]–[15]. It has been demonstrated that convolutional neural networks (CNNs) are highly effective when processing traces with random delay countermeasures [16]. Previous works usually focus on the labels associated with profiled traces, which is undoubtedly the core of neural-network-based SCA. By independently calculating the differences between predictions and target labels, a model is constructed to describe the relationship between the traces and respective labels. However, this process does not explore the connections between traces with the same label or the differences between traces with different labels. Consequently, the features are primarily label-related rather than sample-related. Furthermore, traces from target devices also contain valuable information. As the attack is based on these traces, extracting features from them can significantly improve key recovery. Previous works have mainly focused on the profiled device and rarely utilized these traces for training, leading to a potential loss of valuable information.

Contrastive learning offers a solution to the above problems. It can effectively learn representations by encouraging instances of the same trace, after undergoing different data augmentations, to become more similar. Contrastive learning does not rely on label information; instead, it utilizes the intrinsic features of the traces, enabling the incorporation of traces from the target device into the training process. Thus, we introduce a novel approach called CL-SCA by applying the contrastive learning technique to SCA. This approach captures both similarity and dissimilarity within traces in

Annyu Liu, An Wang, Congming Wei, Yaoling Ding and Liehuang Zhu are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China. (Email:{Annvliu, wanganl, weicm, dyl19, liehuangz}@bit.edu.cn).

Shaofei Sun is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China. (Email:sfsun_shine@bupt.edu.cn).

Yongjuan Wang is with Institute of Cyberspace Security, Information Engineering University, Zhengzhou 450001, China. (Email:pinkywyj@163.com).

(Corresponding author: Shaofei Sun).

Manuscript received April 19, 2021; revised August 16, 2021.

unsupervised scenarios, and then use labeled traces to achieve robust classification results. To the best of our knowledge, this is the first time that a contrastive approach has been used in profiled SCA. By incorporating contrastive learning into SCA, we aim to improve the effectiveness of existing methods.

A. Related Work

Maghrebi *et al.* [11] first introduced neural networks in profiled SCA against the masking countermeasure. Prouff *et al.* [12] introduced the first public dataset named ASCAD, which serves as a common basis for further works on the application of deep learning-based SCA. Zaid *et al.* [17] suggested that a convolutional layer with filter size one enhances feature extraction. In contrast, Wouters *et al.* [13] argued it mainly scales the input and can be replaced with preprocessing methods. Cao *et al.* [14] proposed a bilinear convolutional neural network to address challenges related to masked leakage values. However, these methods focus more on mapping from samples to labels and overlook sample relationships.

Extracting both similarities and dissimilarities within samples has also been explored through techniques such as metric learning and self-supervised learning. Beyond the contrastive learning approach adopted in this study, various other methods, including autoencoders [18], principal component analysis (PCA) [19], mutual information maximization [20], linear discriminant analysis (LDA) [21], and locally linear embedding (LLE) [22], have been explored in the SCA domain under similar learning paradigms. Additionally, Won *et al.* [23] proposed a generic multi-scale CNN framework that integrates multiple preprocessing techniques into deep learning architectures to enhance performance. However, these methods typically separate feature extraction from classification, which can introduce biases and incomplete feature representations. This separation is particularly problematic for neural network classifiers with strong modeling capabilities, as suboptimal feature extraction may negatively impact the subsequent classification process.

Therefore, the motivation of our work is to extract the similarities and dissimilarities between traces, while the extracted features can be further refined in subsequent task-specific training. CL-SCA does not exclude traditional feature extraction methods. Instead, it can improve the feature extraction capability of neural network-based SCA to efficiently recover the secret key of the cryptographic algorithm.

While techniques such as Triplet Networks and Siamese Networks theoretically provide similar capabilities, we prefer contrastive learning due to its simpler and more intuitive logic and design. The use of contrastive learning has gained popularity across various domains, including computer vision and natural language processing. The mature learning frameworks, such as SimCLR [24] and MoCo [25] have emerged. This approach enables the network to gain a deeper understanding of side-channel traces, ultimately improving the overall effectiveness of SCA.

B. Contributions and Organization

The main contributions are summarized as follows:

- We propose a SCA approach based on contrastive learning named CL-SCA. This approach not only concerns label-related features as prior works, but also focuses on the similarity between leakage samples.
- CL-SCA has a high level of generality, and it is not limited to specialized neural network architectures. By integrating CL-SCA into several state-of-the-art works, we illustrate that this approach can further improve performance while inheriting the advantages of the original methods.
- We incorporate traces from the target device into the unsupervised pretraining process to extract features. This integration enables a deeper understanding of the leakage, and further improves the effectiveness of the proposed approach.

The organization of this paper is as follows: Section II introduces the preliminary knowledge. Section III shows CL-SCA in detail, and further proposes CL-SCA+. Section IV describes the experimental results, and Section V presents the conclusion.

II. Preliminaries

A. Profiled Side-Channel Analysis

Profiled SCA leverages the inherent dependence between a cryptographic device's leakage traces and the data it processes. It involves two main stages: a profiled stage and an analysis stage. During the profiled stage, a model is created using leakage traces to represent the secret value processed by the cryptographic device. Then, the secret value of the target device can be revealed by comparing the traces collected from the target device with the model prediction during the analysis stage.

The attacker has access to a profiled device that is similar to the target device during the profiled stage. Using this device, the attacker can select the plaintext p and the secret key k to execute the cryptographic algorithm and collects the corresponding trace t. There is an association between t and the intermediate value y used during the execution of the cryptographic algorithm. For example, in the case of the AES algorithm, the output of the Sbox is usually used as the intermediate value y, and y can be expressed as Eq.(1).

$$y = Sbox(p \oplus k) \tag{1}$$

During the profiled stage of profiled SCA, the model $\varphi(\cdot)$ is established to map the trace to different intermediate values of the cryptographic algorithm. Using the model, the probability of each trace corresponding to different intermediate values y can be calculated, as shown in Eq. (2).

$$\begin{aligned} \varphi(t) &= \Pr[Y = y \mid t] \\ &= \Pr[Y = Sbox(p \oplus k) \mid t] \end{aligned} \tag{2}$$

During the analysis stage, the attacker collects the traces of the target device with different plaintexts. Then the attacker matches each trace with the model $\varphi(\cdot)$ to obtain the probability of different intermediate values $\Pr[Y = y \mid t]$ for different key candidates $k \in \mathcal{K}$, where $\mathcal{K}$ is the set of all possible key candidates $\mathcal{K} = \{0, 1, \cdots, 255\}$. By calculating the result of

Eq.(3), the profiled SCA can identify the key corresponding to the highest log-likelihood estimation score, denoted as k_r.

$$L(k) = \sum_{i=1}^{N} \log \Pr(t_i; k) \tag{3}$$

where N denotes the number of collected traces from the target device. The recovered key k_r is regarded as the key of the target device. The analysis is deemed successful if the use of the recovered key k_r enables correct encryption.

B. Convolutional Neural Network

Profiled SCA can be viewed as a classification problem, which makes it easier to employ deep learning approaches. Among these techniques, CNN has emerged as a widely adopted solution within the SCA community for accurately classifying leakage traces. Fig. 1 shows an example of trace classification using a CNN. Similar to profiled SCA, CNN also calculates the probabilities of each trace corresponding to different intermediate values.

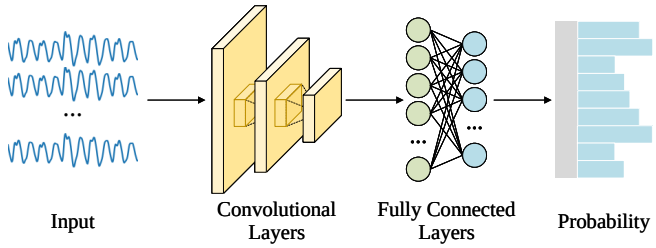


Fig. 1. The schema of CNN.

CNN is typically composed of three main types of layers: convolutional layers, pooling layers, and fully connected layers. The convolutional layer applies filters to the input trace to generate feature maps. Each filter convolves with a small input patch, producing a single value placed in the corresponding position of the output feature map. The convolutional layer produces multiple feature maps representing different filters, which are then passed to the next layer for further processing.

The pooling layer is typically applied after one or more convolutional layers. Its primary purpose is to reduce the spatial dimensions of the feature maps produced by the convolutional layers, while also preserving the most important information. There are various types of pooling operations, but for this paper, we will only focus on average pooling. The output of this layer is the average value of a specific area in the feature map.

Fully connected layers process the entire input volume, unlike convolutional and pooling layers which perform local operations on specific regions of the input data. The input to the fully connected layer usually consists of a flattened feature map that has been produced by one or more preceding convolutional and pooling layers. The flattened feature map is a one-dimensional array of values that represent the features detected in the input trace. The fully connected layer then applies a series of matrix operations to these features to generate a set of output values that correspond to the predicted class probabilities.

C. Contrastive Learning

Contrastive learning is a neural network technology that can learn the representations of data without the need for human-labeled annotations. Its main advantage lies in its ability to extract meaningful representations from unlabeled data. This extraction is achieved by contrasting positive and negative examples during the training process. After training, an encoder $f(\cdot)$ is established to learn a representation space, where similar samples are mapped close together and dissimilar samples are mapped far apart, which satisfies Eq.(4). Fig. 2 illustrates this process.

$$score(f(x), f(x^+)) >> score(f(x), f(x^-)) \tag{4}$$

Here, x^+ represents a positive sample that shares similarities with the sample x, and the pair consisting of x^+ and x is regarded as a positive pair. x^- represents a sample that is dissimilar to x, and it forms a negative pair with x. The $score$ is a measure of similarity between representations, and the function $f(\cdot)$ transforms the samples into their corresponding feature vectors.

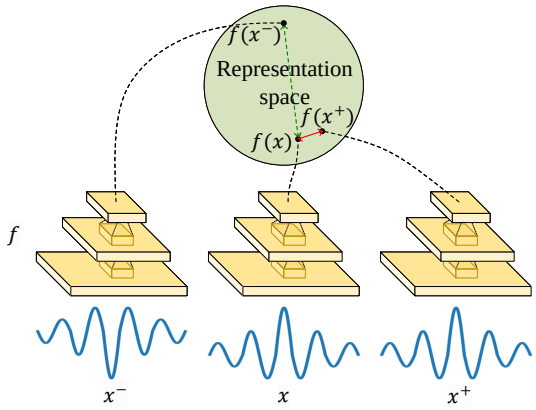


Fig. 2. The process of contrastive learning.

In this paper, a positive pair refers to augmented traces generated from the same original traces, ensuring that the augmented traces are associated with the same intermediate value. The negative samples are the traces augmented from different original traces. To achieve varying augmentation on each trace, we propose a stochastic data augmentation technique and employ it to each trace.

The contrastive loss function is also an important component. We use the normalized temperature-scaled cross entropy loss as loss function, which is named *NT-Xent*. *NT-Xent* takes a pair of traces and their labels as input, and computes a loss that encourages the network to learn representations that separate the positive and negative pairs. The loss function penalizes the distance between the representations of positive pairs, and rewards the distance between the representations of negative pairs.

Overall, contrastive learning is a powerful technique for unsupervised learning, which is applied to SCA in this paper. By learning useful representations in an unsupervised manner, we prove that contrastive learning can help improve the performance of supervised SCA based on machine learning.

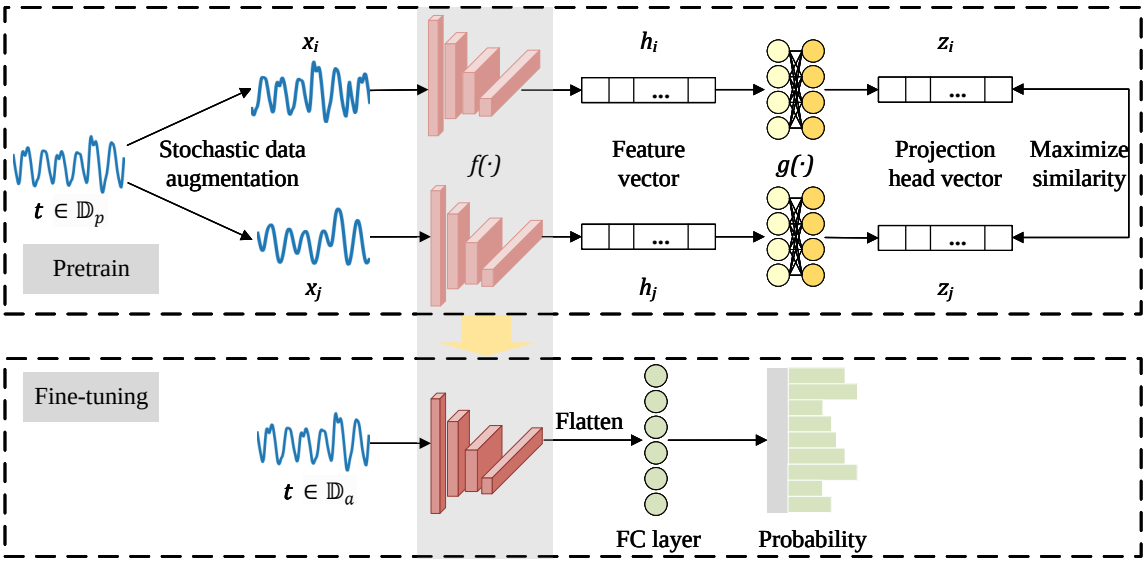


Fig. 3. The basic flow of CL-SCA.

III. Proposed Approaches

In this section, we provide a detailed introduction to our proposed SCA approach. The process of our approach is illustrated in Fig. 3, which includes two distinct parts: pretraining and fine-tuning. By contrasting positive and negative pairs, our approach effectively identifies similarities among leakage traces. The specific details of our proposed approach are described in the following subsections.

A. Stochastic Data Augmentation

Data augmentation has been shown to be an effective technique for improving the capabilities of neural networks [16]. In our proposed approach, the initial step involves applying stochastic data augmentation to each trace. Our goal is to generate two distinct augmented traces from a common original trace t as a positive pair, denoted as x_i and x_j. The aim is to introduce variations in the augmented traces while preserving the valuable features for accurate classification. This enables the encoder to effectively capture the commonalities between the positive pairs during the pretraining part, allowing it to identify the essential features for classification and discard irrelevant information.

To achieve this goal, we incorporate randomness into our data augmentation technique to ensure unpredictability and diversity. As shown in Fig. 4, our stochastic data augmentation technique includes three types of data augmentation: random shift, random cropping, and random denoising. The procedure for each data augmentation technique is as follows:

1) Random Shift (RS): Given the maximum range for left or right shifts L_s, the size of each shift is random. The specific length of movement l_s satisfies the Eq. (5), where l_s is a positive number for a left shift, and a negative number for a right shift.

$$l_s = random(-L_s, L_s) \tag{5}$$

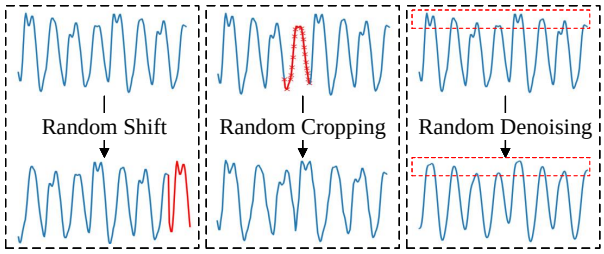


Fig. 4. The illustrations of stochastic data augmentation techniques

2) Random Cropping (RC): Cropping occurs with a probability of 50%. If cropping occurs, the position of the cropping p_c is random and determined by Eq. (6). L_c in this equation denotes the size of the cropping, and L denotes the length of the original trace. After cropping, it is essential to stretch the trace. This involves uniformly filling the cropped trace to ensure that its length remains consistent with its original length.

$$p_c = random(0, L - L_c) \tag{6}$$

3) Random Denoising (RD): Given the maximum window range W, the window size w is random. The denoising procedure is described by Eq. (7), where t_i^k signifies the k-th data point of the original trace t_i, and x_i^j represents the j-th data point of the denoised trace x_i.

$$w = random(1, W), x_i^j = \frac{1}{w} \sum_{k=j-w/2}^{j+w/2} t_i^k \tag{7}$$

B. SCA Approach Based on Contrastive Learning

This section presents the proposed approach for CL-SCA, which consists of two main parts: pretraining and fine-tuning. In the pretraining part, we leverage stochastic data augmentation techniques to effectively capture the valuable

features from the traces. To achieve this, we train the encoder $f(\cdot)$ on N_p traces obtained from the profiled device in an unsupervised setting. It is important to note that these traces and their corresponding labels can form the profiled set $\mathbb{D}_p$. Our encoder $f(\cdot)$ is trained using mini-batch gradient descent. The following is a detailed description of CL-SCA for training:

(1) Perform stochastic data augmentation: Two rounds of stochastic data augmentation are apllied to each trace t_i within a mini-batch, resulting in two augmented traces x_i and x_j. The stochastic data augmentation technique has been discussed in the previous section.

(2) Encode traces: Use the encoder $f(\cdot)$ to encode the augmented traces x_i and x_j into feature vectors h_i and h_j, respectively. By maximizing the similarity between the feature vectors for the positive pair, the encoder is encouraged to focus on the sample-related features, which can improve the accuracy and robustness of subsequent fine-tuning network.

(3) Perform the projection head $g(\cdot)$: Encode the feature vectors h_i and h_j to obtain the projection head output vectors z_i and z_j using the projection head function $g(\cdot)$. The projection head is used during pretraining and discarded afterward. Its effectiveness lies in layer-wise progressive feature weighting and non-linearity, enabling the model to learn more discriminative and transferable features. By introducing the projection head, the model learns a more robust feature representation, reducing over-specialization in the pretraining part. Here, a two-layer MLP is used.

(4) Train the encoder $f(\cdot)$: Calculate the similarity between z_i and z_j as loss function. Then apply backward training to the encoder $f(\cdot)$ to adjust its weights with the objective of maximizing this similarity.

In order to calculate the similarity for a positive pair, we use $\|\cdot\|$ to represent ℓ_2 regularization, and define the cosine similarity of two vectors $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ as:

$$sim(\boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{(\alpha)^T \beta}{\|\alpha\| \|\beta\|}. \tag{8}$$

We define the size of the mini-batch as n. After data augmentation, there are n positive pairs, i.e. $2n$ augmented traces. For a positive pair, we treat $2(n-1)$ augmented traces within a mini-batch as negative examples. Utilizing cosine similarity $sim(\boldsymbol{\alpha}, \boldsymbol{\beta})$ introduced earlier, we define a loss function for the relevant vectors (z_i, z_j) of a positive pair as:

$$\ell(z_i, z_j) = -\log \left(\frac{\exp\left(\frac{sim(z_i, z_j)}{\tau}\right)}{\sum_{k=1}^{2n} \mathbb{I}_{[k \neq i]} \exp\left(\frac{sim(z_i, z_k)}{\tau}\right)} \right), \tag{9}$$

where $\mathbb{I}_{[k \neq i]}$ is an indicator function evaluating to 1 iff $k \neq i$, and τ is set to 0.07 in our work. The numerator in the log calculates the similarity of positive pair (z_i, z_j), while the denominator calculates the similarity of negative examples of z_i. By minimizing this loss, the encoder can extract common valuable features that are still useful for classification after data augmentation.

The final loss function used during pretraining is the average loss of all positive pairs in a mini-batch, as depicted in Eq.(10). This loss function is termed as *NT-Xent*, and has been used in previous contrastive learning researches [26]. We used the same hyperparameters as those studies, as extensive prior work has demonstrated the effectiveness of these parameters.

$$\mathcal{L} = \frac{1}{2n} \sum_{i=1}^{n} [\ell(z_i, z_j) + \ell(z_j, z_i)]. \tag{10}$$

After training multiple epochs in the previous steps, the encoder $f(\cdot)$ can unsupervisedly extract similarities between leakage samples from the profiled set. This part offers a different perspective on feature extraction.

In the fine-tuning part, we utilize the encoder from the pretraining part for profiled SCA. We add a fully connected layer to the trained encoder $f(\cdot)$ to construct the neural network. The process of fine-tuning part is to train this required neural network using the labeled profiled trace set $\mathbb{D}_p$ and evaluate its performance on N_a traces from the target device, which constitute the attack trace set $\mathbb{D}_a$. In general, CL-SCA enables the network to retain the sample-related features extracted during the pretraining part while additionally acquiring label-related features. By incorporating insights from both perspectives, the network achieves better classification results compared to conventional network training approaches.

C. CL-SCA+

While CL-SCA is helpful in extracting crucial features, it still overlooks valuable information present in the attack trace set. To address this limitation, we propose an enhanced approach called CL-SCA+. We incorporate the attack trace set into the training process, allowing us to utilize the untapped information effectively. By integrating the attack traces, CL-SCA+ maximizes the utilization of available data and improves the overall performance of the model.

We observe that the pretraining part of CL-SCA is unsupervised. The pretraining only augments the traces of the profiled set, and then trains the encoder to find the commonalities of the positive pairs. This process does not require any labels of the profiled set. Therefore, we can extend the original input dataset to $\mathbb{D}$, which is composed of the profiled trace set $\mathbb{D}_p$ and the attack trace set $\mathbb{D}_a$ without any labels. Through this, we can enable the model to learn the useful information of the attack trace set during the pretraining part, thereby improving the effectiveness of key recovery.

Fig. 5 illustrates the CL-SCA+ process. This process begins by utilizing unlabeled profiled trace set and attack trace set to pretrain an encoder. This encoder is responsible for extracting the valuable features. Subsequently, a neural network is constructed based on this pretrained encoder and fine-tuned using a labeled profiled trace set. Finally, the fine-tuned neural network is employed to classify the traces present in the attack trace set, ultimately leading to the key recovery.

IV. EXPERIMENTS

To evaluate the effectiveness of the proposed approach, we conduct experiments in this section. We conduct the experiments using the PyTorch library in Python on a workstation equipped with an Nvidia GTX 3090 GPU. The hyperparameters used in pretraining part for all experiments are provided

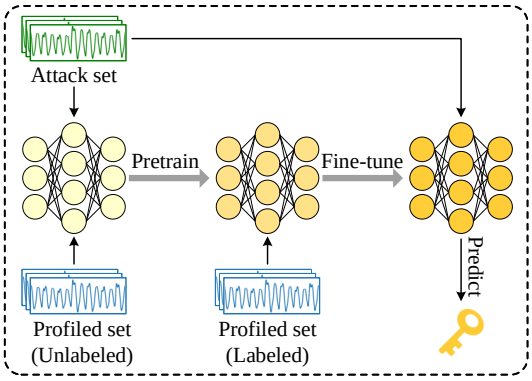


Fig. 5. The process of CL-SCA+.

in Table I. Guessing entropy (GE) is used as an evaluation metric, which is widely accepted as the evaluation criterion in the SCA field. It quantifies the ranking of the correct key among all candidate keys. Therefore, a lower GE indicates a more accurate key recovery. To obtain robust results, we shuffle the order of the attack traces 100 times for each model and calculate the average GE curve as the result.

TABLE I
PRETRAINING HYPERPARAMETERS

Petrain Hyperparameters	
Optimizer	Adam
Weight Decay	0.0001
Learning Rate	0.00001
Loss Function	*NT-Xent*
τ in *NT-Xent*	0.07
Epochs	200
Projection Head Output	128

A. Datasets

Datasets used in our experiments are shown in Table II, covering various platforms, implementations, and countermeasures. Our target is to thoroughly evaluate the generalizability of the proposed approach.

TABLE II
INFORMATION ABOUT DATASETS

Dataset	**Implementation**	**SCA Constrains**
ASCAD$^{\text{sync}}$	AVR architecture ATMega8515	First-order mask
ASCAD$^{\text{desync100}}$	AVR architecture ATMega8515	First-order mask & Desynchronization
AES-SAKURA	SAKURA-G platform	Low SNR
RSA	SAKURA-G platform	Atomic Version & First-order mask & Low SNR
FESH	SAKURA-G platform	First-order mask & Low SNR

(1) ASCAD dataset: ASCAD [12] is a public benchmark dataset for evaluating neural network-based SCA techniques. The dataset contains electromagnetic traces from a protected AES implementation on an ATMega8515 development board, featuring various desynchronization levels. Each variant includes 60000 traces: 50000 for profiling and 10000 for attacks. Our evaluation focuses on ASCAD$^{\text{sync}}$ (no desynchronization) and ASCAD$^{\text{desync100}}$ (desynchronization level of 100).

(2) AES-SAKURA dataset: The AES-SAKURA dataset contains electromagnetic traces from the AES-128 algorithm running on the SAKURA-G platform. This hardware platform results in relatively low signal-to-noise (SNR) leakage, which makes SCA more challenging. We collected these traces using a specialized H-field probe, a low-noise amplifier with a gain of 30 dB, and the Keysight MSOS054A oscilloscope at a sampling rate of 500 MSa/s. The dataset focuses on the input of the first byte in the last round S-box. A total of 44,200 traces were collected, comprising 35,000 labeled traces and 9,200 unlabeled traces.

(3) RSA dataset: The RSA dataset comes from an atomic RSA implementation on the SAKURA-G platform using a 32-bit mask. We categorized the RSA traces into segments based on modular multiplication and squaring. 50000 trace segments were collected, including 40000 labeled and 10000 unlabeled segments.

(4) FESH dataset: FESH is a lightweight block cipher that is designed for both security and efficiency, making it ideal for resource-constrained environments such as IoT devices. The FESH dataset records power consumption from the first-order masking implementation on the SAKURA-G platform, with a sampling rate of 125 MSa/s. Each trace consists of 1000 data points from the S-box of the first round of FESH, providing the first 4-bit output information. The dataset includes a total of 15000 traces, comprising 5000 labeled traces and 10000 unlabeled traces.

B. Results of CL-SCA

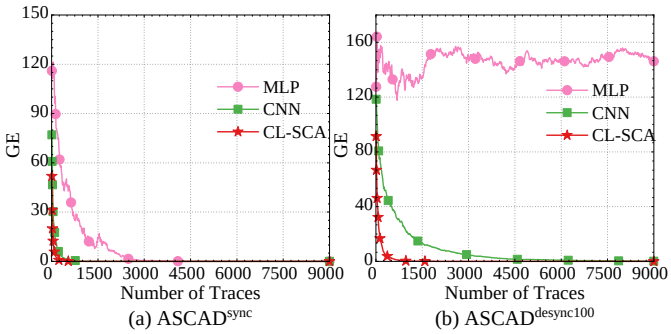
The ASCAD is a widely used dataset in the field of side-channel analysis. While many studies have examined its leakage patterns [27], [28], there remains potential to enhance networks' performance in exploiting these vulnerabilities. First, the performance of CL-SCA is evaluated on the two ASCAD datasets. The network architecture is described in [12] and the fine-tuning hyperparameters for ASCAD$^{\text{sync}}$ are shown in Table III.

TABLE III
FINE-TUNING HYPERPARAMETERS FOR ASCAD

Network Architecture	
Convolution Blocks	5
Filters	{64, 128, 256, 512, 512}
Convolutional Layer Kernel Size	11
Activation Function	ReLU
Pooling Layer	Average
Pooling Layer Kernel Size	2
Fully Connected Layers	2
Neurons	4096
Fine-tuning Hyperparameters	
Batch Size	200
Optimizer	RMSprop
Learning Rate	0.00001
Loss Function	Cross Entropy Loss

The performance of CL-SCA is compared with that of MLP and CNN, which use the same model architecture as CL-

SCA. We train 10 models for CL-SCA, CNN and MLP and calculated the average performance. The number of required attack traces is denoted as N_{GE}. Fig. 6 and Table IV shows the results on the ASCAD$^{\text{sync}}$ and ASCAD$^{\text{desync100}}$. Notably, MLP exhibits the worst performance, while CL-SCA outperforms CNN on both datasets. In the case of ASCAD$^{\text{sync}}$, CL-SCA only requires 828 attack traces, while CNN requires 1156 traces and MLP requires over 2500 traces. This significant improvement achieved through contrastive learning results in a reduction of 28.37% for CNN and an impressive 81.22% for MLP. For ASCAD$^{\text{desync100}}$, MLP fails to recover the key, and CL-SCA outperforms both MLP and CNN. While CNN requires 8577 traces, CL-SCA achieves the key recovery with only 2540 traces, representing an improvement proportion of 70.39%. These remarkable advancements underscore the superior utilization of supervised data by CL-SCA. By extracting similarities among augmented traces and identifying differences between negative pairs, the model successfully captures a wealth of valuable information and features.



(a) ASCAD$^{\text{sync}}$ (b) ASCAD$^{\text{desync100}}$

Fig. 6. GE for different ASCAD dataset.

TABLE IV
THE N_{GE} OF DIFFERENT APPROAHES

Dataset	MLP [12]	CNN [12]	CL-SCA
ASCAD$^{\text{sync}}$	4410	1156	828
ASCAD$^{\text{desync100}}$	\	8577	2540

We conduct CL-SCA, CNN and MLP on AES-SAKURA dataset, and present the results in the Fig. 7. The network architecture and hyperparameters of fine-tuning part are similar to those of ASCAD. The only difference lies in the inclusion of a batch normalization layer within each convolutional block. We set the epoch to 30 to avoid overfitting. Due to low SNR resulting from the simultaneous operation of 16 Sboxes, the experimental results are worse than ASCAD. However, CL-SCA still exhibited significant advantages over CNN and MLP, even when applied to completely different hardware parallel AES implementations. Moreover, by utilizing CL-SCA, we were able to achieve key recovery through key enumeration. This indicates that the improvements brought by contrastive learning are generalizable, repeatable, and not limited to a specific implementation.

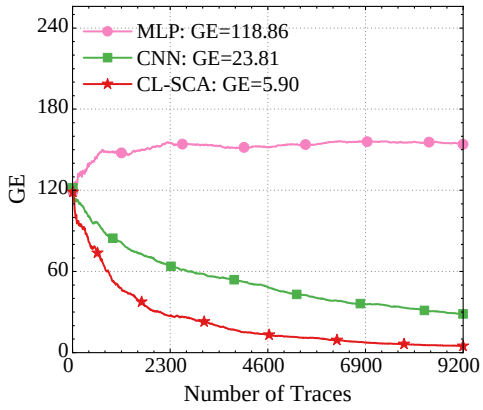


Fig. 7. GE for the AES-SAKURA dataset.

C. Experiments about Data Augmentation

As a valuable technique for generating positive samples, stochastic data augmentation undoubtedly has an impact on CL-SCA. In this subsection, we conduct experiments on three datasets to explore the impact of different combinations of data augmentation on CL-SCA. We first experimented with different data augmentations on ASCAD$^{\text{sync}}$ while using a CNN as a baseline. For each configuration, we trained 10 models and then calculated the average performance. Table V provides a comparison of the results.

TABLE V
THE PERFORMANCE OF STOCHASTIC DATA AUGMENTATION COMBINATIONS ON ASCAD$^{\text{SYNC}}$

Number of Types	Types	Required Traces
None	\	1156
One type	RC	1279
	RD	**526**
	RS	707
Two types	**RD + RS**	**688**
	RD + RC	1180
	RS + RC	743
Three types	**RD + RS + RC**	**828**

In the ASCAD$^{\text{sync}}$ dataset, the CL-SCA method combined with RD achieves the best performance, requiring approximately 526 traces for successful key recovery. The combination of CL-SCA with RD + RS is the second most effective technique. The addition of RS, when used with convolutional layers, helps the model avoid focusing on specific parts of the trace, allowing it to consider relationships between different samples instead. Moreover, RS enhances the SNR, leading to improved results. In contrast, the use of RC provides more limited improvement, as it discards useful segmentations that are crucial for extracting commonalities. When all three data augmentation techniques are employed, 828 traces are needed for successful key recovery. Although CL-SCA with all augmentations still outperforms CNN, it does not achieve the highest performance in every experiment. The experiments

on ASCAD^{sync} highlight the importance of selecting data augmentation strategies that are well-suited to the specific characteristics of the dataset. Compared to CNN, CL-SCA with RD reduces the number of traces needed for key recovery by 630, demonstrating a 54.5% reduction in trace count.

We conducted the same experiments on ASCAD^{desync100} to identify the optimal combination of data augmentations. The results are presented in Table VI. Among the augmentation techniques tested, RS achieved the best performance, while RC also delivered promising results. This improvement can be attributed to their capacity to introduce offsets, allowing the network to better extract features from unsynchronized traces. However, the findings suggest that using multiple augmentations does not always lead to enhanced performance; in fact, excessive modifications can obscure the asynchrony patterns that are essential for effective feature learning.

TABLE VI
THE PERFORMANCE OF STOCHASTIC DATA AUGMENTATION COMBINATIONS ON ASCAD^{DESYNC100}

Number of Types	Types	Required Traces
None	\	8577
One type	RC	2399
	RD	2463
	RS	**1828**
Two types	**RD + RS**	**2207**
	RD + RC	3036
	RS + RC	2535
Three types	**RD + RS + RC**	**2540**

The performance of CL-SCA varies depending on the augmentation strategy, with RC reducing its effectiveness when applied alongside other techniques. Compared to ASCAD^{sync}, the experiments on ASCAD^{desync100} reveal the challenges introduced by countermeasures, as they significantly increase the number of traces required for a successful attack. Despite this, RS remains the most effective augmentation under desynchronization, reinforcing the idea that targeted augmentation strategies can enhance attack performance when countermeasures in cryptographic implementations are well understood.

To determine the most suitable data augmentation strategy for different AES implementations, we conducted experiments on the AES-SAKURA dataset, with results presented in Table VII. Among individual augmentation techniques, CL-SCA with RD shows only a slight improvement over CNN, whereas RC significantly enhances performance. This is attributed to the characteristics of hardware implementations, where the SNR is highest at peak positions and gradually declines elsewhere. By effectively removing low-SNR points, RC improves the model's ability to extract meaningful features from traces.

In contrast, the combination of RS and RD, while yielding competitive results, does not outperform RS alone, suggesting that the interplay between augmentation techniques can sometimes diminish effectiveness rather than enhance it. More generally, while multiple augmentation techniques still outperform CNN, their benefits are not necessarily additive. Although CL-SCA does not reduce GE to 0 within 9200 traces, key enumeration enables successful key recovery. These findings reaffirm that tailoring data augmentation strategies to the specific characteristics of the dataset is crucial for optimizing CL-SCA performance.

TABLE VII
THE PERFORMANCE OF STOCHASTIC DATA AUGMENTATION COMBINATIONS ON AES-SAKURA

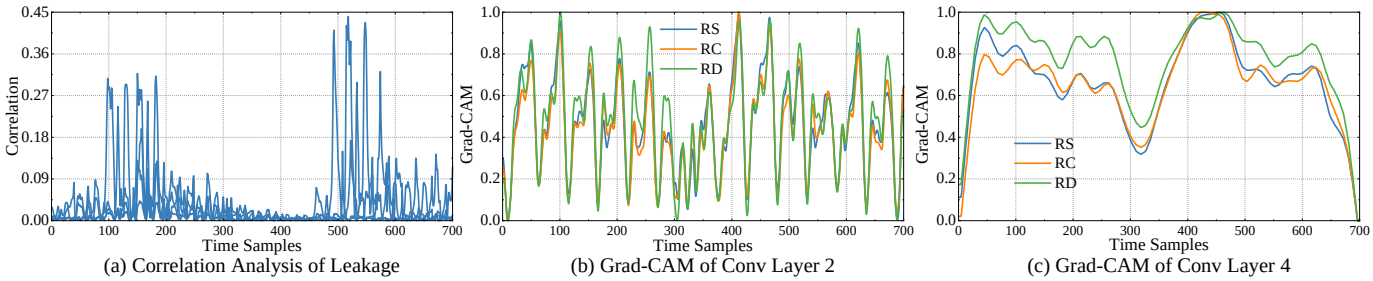
Number of Types	Types	GE
None	\	23.81
One type	**RC**	**4.56**
	RD	21.65
	RS	8.77
Two types	**RD + RS**	**5.50**
	RD + RC	14.81
	RS + RC	17.72
Three types	**RD + RS + RC**	**5.90**

Since stochastic data augmentation is proved to be effective in key recovery, Gradient-weighted Class Activation Mapping (Grad-CAM) [29] is used to visualize its impact, showing the valuable regions and key features the model focuses on during training. Grad-CAM is applied to second and forth convolutional layers of the CL-SCA model trained with RS, RC, and RD on the ASCAD^{sync}. Results are normalized and plotted in Fig. 8(b) and (c). Additionally, a correlation analysis between leaked values and traces is provided in Fig. 8(a). The Grad-CAM of forth convolutional layer highlights leakage locations that are generally broader than those detected by correlation analysis. This discrepancy arises because convolution integrates effective information with surrounding context, expanding the range of useful information.

As observed in the results, CL-SCA with RD, which performs best, exhibits the highest Grad-CAM values across both convolutional layers. In particular, the fourth convolutional layer highlights its ability to capture high-noise features at positions 200–300 and 550–650. In contrast, CL-SCA with RS and RC exhibit consistently lower Grad-CAM values in the second convolutional layer. Meanwhile, in the fourth convolutional layer, Grad-CAM further reflects reduced network focus in low SNR regions. This suggests that proper augmentation in CL-SCA enhances feature extraction, improving the ability to capture hidden information from traces.

D. Experiments for CL-SCA+

Although the previous subsection demonstrates the advantages of CL-SCA, this approach still has limitations as it does not utilize information from the attack trace set, despite the presence of useful information in the attack trace set. To address this issue, we utilize the useful information in the attack trace set during the pretraining part of CL-SCA and propose an enhanced approach called CL-SCA+. This approach enables the model to learn some attack trace set's information in an unsupervised manner. In order to evaluate



(a) Correlation Analysis of Leakage (b) Grad-CAM of Conv Layer 2 (c) Grad-CAM of Conv Layer 4

Fig. 8. Leakage evaluation and Grad-CAM visualization of network focus.

the effectiveness of the enhanced approach, we will conduct a performance comparison among three different approaches: CNN, CL-SCA and CL-SCA+. The experimental setup is shown in Table VIII, and the hyperparameters of networks are the same as those used in earlier experiments.

TABLE VIII
EXPERIMENT SETUP

	CNN	CL-SCA	CL-SCA+
Pretrain with stochastic data augmentation	×	✓	✓
Pretraining set	×	Profiled Traces	Profiled Traces and Attack Traces
Fine-tuning/ Regular training	✓	✓	✓
Training set	Profiled Traces	Profiled Traces	Profiled Traces

We first conduct the experiments on ASCAD$^{\text{sync}}$, and the results are shown in Fig. 9. Shadows of different colors represent the GE range of multiple experiments, while different dark curves represent the average GE of these experiments. It can be observed that the CL-SCA has significant advantages over the CNN. Several CNN experiments show that 800 attack traces are not sufficient to reduce GE to 0, whereas all experiments of the CL-SCA can reduce GE to 0. This suggests that using contrastive learning and capture similarity between augmented traces in pretraining part can help the model focus on previously overlooked information, significantly improving its effectiveness.

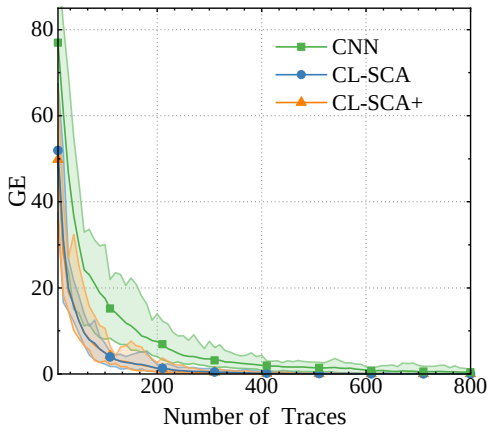


Fig. 9. Distribution of multiple experimental results of CL-SCA and CL-SCA+ on ASCAD$^{\text{sync}}$.

The results of CL-SCA and CL-SCA+ exhibit a close proximity, with an insignificant difference between them as shown in Fig. 9. Therefore, we analyze the results from all experiments of these two approaches in more detail. We calculate the distribution of the number of required attack traces separately, as shown in Fig. 10. The horizontal axis represents the range of the number of traces required to reduce GE to 0, while the vertical axis represents the number of experiments. In the case of successful secret key recovery in CL-SCA+, we have observed that the maximum number of experiments occurs when the number of attack traces falls within the range of (450, 500]. In contrast, when it comes to CL-SCA, the range of (500, 550] attack traces witnessed the highest number of experiments. When processing the same amount of traces, it is more likely for CL-SCA+ to successfully recover the key compared to CL-SCA. We also calculate the average number of traces required for all experiments and find that CL-SCA requires 526 attack traces when the secret key is successfully recovered, while CL-SCA+ requires 518 traces. This indicates that utilizing the attack trace set in the pretraining part can help the model learns more useful information and improves key recovery performance.

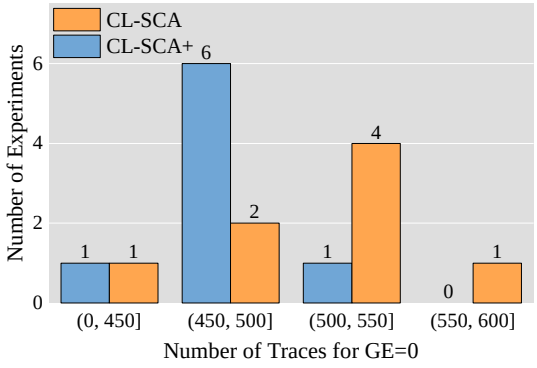


Fig. 10. Distribution of multiple experimental results of CL-SCA and CL-SCA+ on ASCAD$^{\text{sync}}$.

We conduct the same experiments on ASCAD$^{\text{desync100}}$, and the results are depicted in Fig.11. 2000 attack traces are insufficient for CNN to recover the key. Similar to the experiments on ASCAD$^{\text{sync}}$, CL-SCA and CL-SCA+ show comparable results. Therefore, we conducted a more detailed analysis of the results from all experiments using these two approaches. Fig. 12 illustrates the results of each experiment. Clearly, in terms of the distribution of the required number of traces, it

is obvious that CL-SCA+ outperforms CL-SCA. The average number of traces required for CL-SCA is 1828, while CL-SCA+ requires 1777 traces. This observed difference in the mean values of multiple experiments further reinforces the superiority of CL-SCA+. Consequently, the introduction of the attack set enhances the effectiveness of the proposed approach.

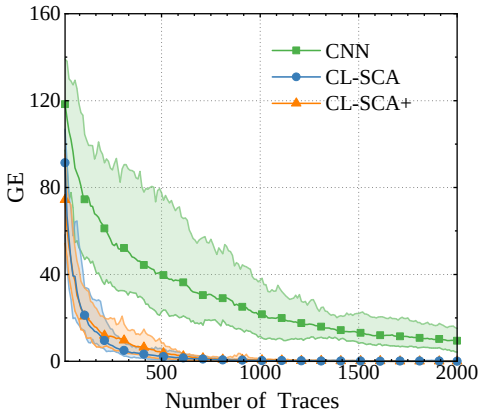


Fig. 11. The results of CL-SCA and CL-SCA+ on ASCAD$^{\text{desync100}}$.

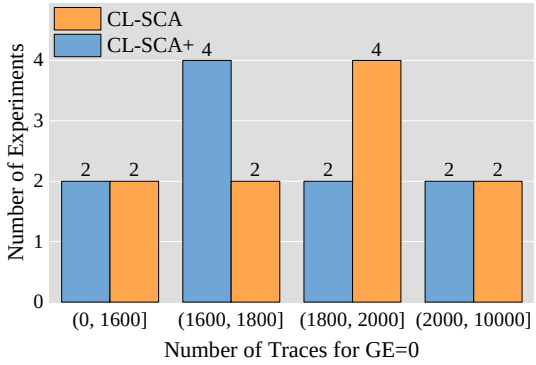


Fig. 12. Distribution of multiple experimental results of CL-SCA and CL-SCA+ on ASCAD$^{\text{desync100}}$.

We finally conduct the experiments on AES-SAKURA dataset, and the results are presented in Fig. 13. Both CL-SCA methods are significantly superior to CNN, and the difference between CL-SCA+ and CL-SCA is more pronounced in AES-SAKURA dataset. Therefore, we don't conduct a more detailed analysis of the results as before. The results of improved CL-SCA+ are more stable, and its GE is lower. These experimental results once again highlight the benefits that contrastive learning bring to SCA, and the utilization of attack trace sets can also improve the effectiveness of key recovery.

E. Experiments on Profiled Set

To further explore the impact of introducing the attack set, we consider a more challenging attack scenario when the attacker only has a limited number of profiled traces. Fortunately, CL-SCA+ can completely use all traces, regardless of whether they are labeled or not. This advantage enables CL-SCA+ to outperform CNN and CL-SCA when the availability

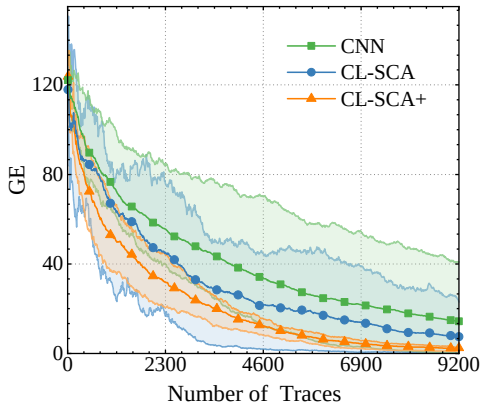


Fig. 13. The results of CL-SCA and CL-SCA+ on AES-SAKURA.

of labels is limited. We explore the impact of using different number of labels, and compare performance of CL-SCA, CL-SCA+ and CNN on the different datasets.

Firstly, we conduct experiments on ASCAD$^{\text{sync}}$. All 60000 traces are utilized, and the number of profiled traces is incrementally increased in steps of 10000 from 10000 to 50000. Let N_p represent the number of profiled traces. The average results are presented in Table IX. It can be observed that as the number of profiled traces increases, the N_{GE} of all three methods gradually decreases. Moreover, CL-SCA+ outperforms CL-SCA, and both outperform CNN.

TABLE IX
EVOLUTION OF N_{GE} DEPENDING ON N_p ON ASCAD

N_{GE} \ N_p		10000	20000	30000	40000	50000
ASCAD$^{\text{sync}}$	CNN	6312	3498	2438	1411	1156
	CL-SCA	4493	1803	953	666	526
	CL-SCA+	4120	1668	931	583	518
ASCAD$^{\text{desync100}}$	CNN	\	\	20847	13556	8577
	CL-SCA	34190	9626	4508	2648	1828
	CL-SCA+	26404	6921	4322	2542	1777

Additionally, we also analyzed the reduction in the number and proportion of N_{GE}. The smaller the size of the profiled set, the greater peformance improvement brought by both CL-SCA+ and CL-SCA. This indicates that CL-SCA and CL-SCA+ are able to overcome the limitations imposed by the availability of profiled labels. What's even more interesting is that when using fewer profiled traces, the difference in N_{GE} between CL-SCA+ and CL-SCA becomes more pronounced. This is because as the number of profiled traces decreases, the information extracted from them becomes limited. In such cases, the advantage gained from introducing the attack set becomes more prominent. Moreover, the reduced proportion of attack traces in CL-SCA+ remains roughly constant (around 50% compared to CNN) when altering the number of profiled traces. This stability demonstrates that the improvements achieved by CL-SCA+ are consistent and unaffected by the number of profiled traces.

Secondly, we conducted experiments on ASCAD$^{\text{desync100}}$ using the same experimental configuration, and the results

are also shown in Table IX. In the case of profiling with no more than 20000 traces, CNN can not recover the key within 40000 attack traces, while CL-SCA and CL-SCA+ optimize the results of the same network architecture and successfully recover the key. For profiling with 30000 to 50000 traces, CL-SCA and CL-SCA+ show significant improvements compared to CNN. The proportion of reduced traces is noticeably higher than that in ASCAD$^{\text{sync}}$, indicating that contrastive learning brings more pronounced improvements for datasets which are more challenging and require a larger number of attack traces. Furthermore, CL-SCA+ builds upon CL-SCA through the introduction of the attack set. The performance difference is more pronounced when there are fewer profiled traces, while it becomes smaller when profiled traces are sufficient and the introduction of the test set has less impact.

Finally, we conduct experiments on AES-SAKURA dataset. We use all 44,200 traces, and vary the number of profiled traces from 10,000 to 35,000. As shown in Table X, the low SNR of these traces renders the current number insufficient for successful key recovery. The increase in profiled traces not only optimizes the network but also reduces the attack traces. Nonetheless, CL-SCA+ performs the best, followed by CL-SCA, and CNN performs worst.

TABLE X
EVOLUTION OF GE DEPENDING ON N_p ON AES-SAKURA

GE \ N_p	10000	20000	30000	35000
CNN	33.20	12.18	19.59	23.81
CL-SCA	30.71	6.69	8.06	4.56
CL-SCA+	10.22	3.27	7.63	3.95

We conduct a statistical analysis to evaluate the optimizations achieved by our approaches. The increase in the number of profiled traces results in greater optimization for CL-SCA, illustrating that commonality extraction enables the network to learn more features. Conversely, reducing the number of attack traces limits the improvement of CL-SCA+ compared to CL-SCA. However, the improvement of CL-SCA+ keeps more stable. This stability indicates that using all traces in pretraining part motivates full feature extraction, without being constrained by the number of profiled labels.

F. Comparison with the State-of-the-art Works

After illustrating the performance of our approaches, we integrate them with several existing research studies. As mentioned section III, our approaches do not require a special architecture, making them suitable for various networks. In this section, we apply our approaches to several prior research studies , and demonstrate that they can further improve performance while inheriting original advantages of network architecture. Since section IV-D and section IV-E have already proved the the superiority of CL-SCA+ over CL-SCA from multiple perspectives, we focus solely on comparing CL-SCA+ with the prior research studies.

We select several representative works, and implement their approaches in our experimental environment to ensure fairness. These studies have exhibited stable performance, ensuring that the average results from multiple experiments match the findings reported in the respective papers. All other hyperparameters, including network architectures, optimizer, loss function and more, remain the same in both CL-SCA+ and the original approaches.

The experimental results are presented in Table XI. For each experimental setup, we execute the experiments 10 times and calculate the average number of required attack traces. For ASCAD$^{\text{sync}}$, the results show that incorporating the pretraining part leads to at least 8.5% reduction in the number of required attack traces. As mentioned earlier, similarity extraction and the introduction of attack set in the pretraining part effectively improve the effectiveness of key recovery. Additionally, CL-SCA+ exhibits the versatility to be applied to various network architectures and hyperparameters, further improving their performance when keeping original advantages.

TABLE XI
N_{GE} COMPARED TO PRIOR STUDIES ON ASCAD$^{\text{SYNC}}$

Experiment Setup			[12]	[13]	[14]
ASCAD$^{\text{sync}}$	Original approach	N_{GE}	1156	236	101
		Complexity	66653944	6436	17852
	CL-SCA+	N_{GE}	518	216	90
		Complexity	82910840	5138	16554
ASCAD$^{\text{desync100}}$	Original approach	N_{GE}	8577	337	176
		Complexity	66653944	42268	134076
	CL-SCA+	N_{GE}	1828	332	124
		Complexity	82910840	40000	132778

For ASCAD$^{\text{desync100}}$, CL-SCA+ continues to exhibit notable improvements compared to the original approaches. In the case of the approach from reference [13], the improvement is relatively small. This is attributed to the noticeable fluctuations in the required number of traces for original approach, which also affects the effectiveness of CL-SCA+. However, for the current state-of-the-art method, B-CNN [14], CL-SCA+ achieves further substantial performance optimization.

We also analyzed the complexity of different approaches. As the model architecture of fine-tuning part is the same as the original approaches, we focused only on the complexity of the pretraining part of CL-SCA+. From the Table XI, it can be observed that the complexity of CL-SCA+'s model is comparable to that of the original approaches. This is because we use a feature vector dimension of 128 for encoding, which is smaller than the number of classes. Therefore, replacing the classification layer with a projection head often does not significantly increase the complexity of the approaches. This implies that CL-SCA+ does not make the approaches more complex or difficult to train. Moreover, CL-SCA+ can also yield substantial improvements in terms of effectiveness.

G. Discussion

To justify the overall architecture of CL-SCA, experiments are conducted to evaluate the contributions and the computational complexity of each component. In order to make the comparison more obvious, the network structure proposed in

[12] is used, and experiments are performed on two ASCAD datasets.

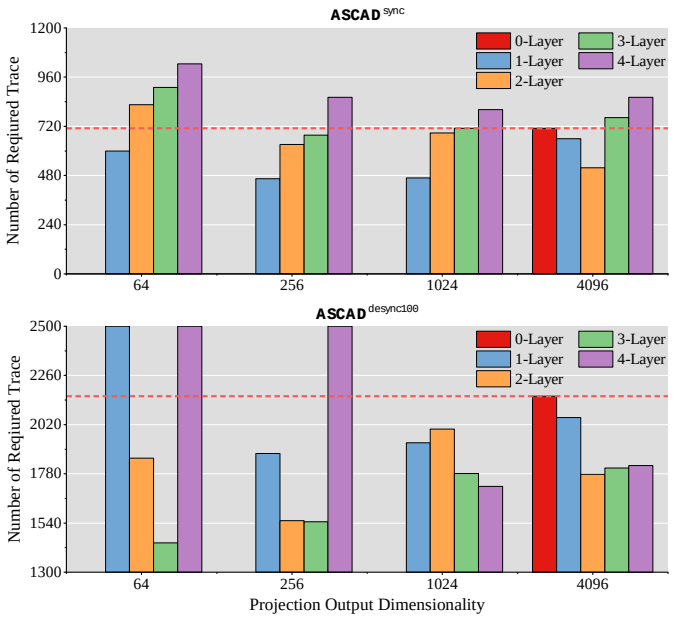


Fig. 14. IMPACT OF PROJECTION HEAD CONSTRUCTION ON N_{GE}

To evaluate the impact of the projection head, N_{GE} values for different projection head configurations are presented in Fig. 14, where '0-layer' indicates that this component is not used. The results show that the performance generally deteriorates as the number of projection head layers increases. On the ASCAD$^{\text{desync100}}$ dataset, the performance is particularly poor with 1-layer and 4-layer projection heads. It demonstrates that increasing the depth of projection heads does not necessarily improve performance, whereas excessively low-dimensional projections result in the loss of valuable information. Therefore, for CL-SCA, the projection head should consist of 2 or 3 layers, with the projection head dimension ranging from 256 to 1024, which yields better performance compared to traditional neural network-based SCA.

To investigate the impact of positive and negative pairs on the experimental outcomes, N_{GE} of different batch size are evaluated. The results in Table XII show that excessively large batch sizes degrade performance, with N_{GE} increasing significantly when the batch size exceeds 400. For ASCAD$^{\text{sync}}$, a batch size of 200 achieves the best performance, while for ASCAD$^{\text{desync100}}$, the optimal batch size is 100. Smaller batch sizes still perform well, but overly large batches may cause rapid data traversal and increased negative pair mismatches, leading to suboptimal convergence. A moderate batch size between 100 and 200 is recommended, while batch sizes above 400 should be avoided to prevent performance degradation.

TABLE XII
IMPACT OF PRETRAINING BATCH SIZE ON N_{GE}

Batch Size	50	100	200	400	600
ASCAD$^{\text{sync}}$	531	527	518	712	677
ASCAD$^{\text{desync100}}$	1682	1599	1777	2320	2325

To evaluate the cost of CL-SCA, the time and space requirements for each component are examined. The statistical results are presented in Table XIII. Since the process for fine-tuning is nearly identical to that of traditional CNNs, only the cost statistics for fine-tuning are presented here. The more complex loss calculation and the addition of the projection head result in at least a 50% increase in both forward and backward pass time. Additionally, since each sample undergoes stochastic data augmentation twice, the data transfer time has doubled. The increase in memory usage has not been as dramatic as the time overhead. This indicates that while CL-SCA may not necessarily outperform other ML methods in terms of time consumption, its storage requirements on resource-constrained hardware remain comparable to those of CNNs. Considering the performance improvements brought by CL-SCA, its computational cost is acceptable, making it overall superior to traditional ML and CNN approaches.

TABLE XIII
TIME AND SPACE COST STATISTICS ON ASCAD[1]

Components		Part Time	Total Time	Space RAM	Space VRAM
Pretrain Part	Data Augmentation	307.379s	1903.089s	6526MB	32126MB
	Loss Calculation	114.780s			
	Model Forward	200.967s			
	Model Backward	352.731s			
	Others[2]	927.232s			
Fine-tuning Part	Copy model	0.422s	843.413s	4979MB	31983MB
	Model Forward	118.716s			
	Model Backward	230.835s			
	Others	493.440s			

1 The experimental setup includes a server equipped with an A100 GPU, an Intel Xeon Gold 6230 processor, and 256GB of memory.
2 'Others' includes operations such as data preparation, result storage, and data transfer between CUDA and the CPU.

To evaluate the generalizability of CL-SCA, experiments are conducted on the RSA and FESH datasets. The performance of different methods on the RSA dataset is presented in Fig. 15(a), with numbers indicating average accuracy. CL-SCA achieves higher peak accuracy and more stable performance compared to CNN, while CL-SCA+ further improves accuracy. Since RSA implementations are inherently more complex than masked AES implementations, their traces exhibit more obvious leakage patterns. Consequently, CL-SCA is sufficient for effective feature extraction in such scenarios. These results suggest that CL-SCA enhances the model's ability to understand asymmetric cryptographic traces and extract generalized representations. The results on the FESH dataset are shown in Fig. 15(b). CL-SCA reduces the N_{GE} from 7571 to 6416, achieving a 15.26% improvement over CNN. CL-SCA+ further lowers N_{GE} to 6395, demonstrating its effectiveness in lightweight cryptographic implementations.

The effect of stochastic data augmentation on these two datasets is evaluated and shown in the third column of Table XIV. The RSA results indicate that the combination of RS and RC achieves the highest accuracy, improving by 5.155% over the traditional CNN method and significantly reducing brute-force complexity. Thus, CL-SCA demonstrates advantages in asymmetric cryptographic algorithms. The fourth column of

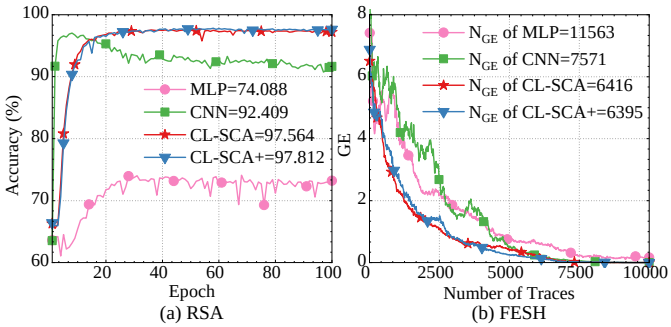


Fig. 15. Accuracy for the RSA dataset and GE for the FESH dataset.

Table XIV presents the results for the FESH dataset, where CL-SCA achieves comparable GE values across different stochastic data augmentation techniques. The best performance is observed when all techniques are applied, reducing N_{GE} for a successful attack by 15.26% compared to the traditional CNN method. This highlights CL-SCA's effectiveness in lightweight cryptographic algorithms.

TABLE XIV
THE PERFORMANCE OF STOCHASTIC DATA AUGMENTATION COMBINATIONS ON RSA AND FESH DATASET

Number of Types	Types	Accuracy(%) on RSA	Required Traces on FESH
None	\	92.409	7571
One type	RC	89.457	**6572**
	RD	**95.085**	6936
	RS	83.955	6747
Two types	RD + RS	82.652	6538
	RD + RC	94.729	6454
	RS + RC	**97.564**	**6452**
Three types	RD + RS + RC	**96.801**	**6416**

V. Conclusion

In this paper, we present an innovative approach called CL-SCA, which introduces contrastive learning techniques to SCA. Unlike previous methods, our approach not only focuses on capturing label-related features in the traces but also extracts similarities among them. This comprehensive feature extraction from the profiled traces leads to a significant improvement in the classification performance of the network. One of the key advantages of CL-SCA is its generality, as it can be applied to various neural network-based SCA approaches without being restricted to a specific network architecture. This versatility enhances the effectiveness of existing SCA methods based on neural networks, while preserving their inherent advantages.

To further enhance the capabilities of CL-SCA, we propose an extension called CL-SCA+ that incorporates the attack set into the training process. This improved approach can unsupervisedly extract valuable information from the attack trace set, further enhancing its performance. Experimental results consistently demonstrate that our approaches outperform other existing methods. Additionally, we investigate the impact of data augmentation on the performance of our approach, providing insights into further improving its effectiveness.

In the future, we intend to apply the concept of contrastive learning to various research findings in SCA methods and investigate its ability for enhancing these approaches. In addition, we recognize the need for further exploration into the impact of different components within CL-SCA, such as the projection head and network structure, on experimental results. This exploration will better assist us in understanding which information contrastive learning specifically extracts from leakage.

Acknowledgments

This work was supported in part by National Key R&D Plan of China (2022YFB3103800), National Natural Science Foundation of China (62272047, 62302036, 62402039), Beijing Natural Science Foundation (L244044, QY24173).

References

[1] M. Zhao and G. E. Suh, "Fpga-based remote power side-channel attacks," in *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*. IEEE Computer Society, 2018, pp. 229–244. [Online]. Available: https://doi.org/10.1109/SP.2018.00049

[2] F. Schellenberg, D. R. E. Gnad, A. Moradi, and M. B. Tahoori, "An inside job: Remote power analysis attacks on fpgas," *IEEE Des. Test*, vol. 38, no. 3, pp. 58–66, 2021. [Online]. Available: https://doi.org/10.1109/MDAT.2021.3063306

[3] Y. Zhang, R. Yasaei, H. Chen, Z. Li, and M. A. A. Faruque, "Stealing neural network structure through remote FPGA side-channel analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4377–4388, 2021. [Online]. Available: https://doi.org/10.1109/TIFS.2021.3106169

[4] M. C. Martínez-Rodríguez, I. M. Delgado-Lozano, and B. B. Brumley, "Sok: Remote power analysis," in *ARES 2021: The 16th International Conference on Availability, Reliability and Security, Vienna, Austria, August 17-20, 2021*, D. Reinhardt and T. Müller, Eds. ACM, 2021, pp. 7:1–7:12. [Online]. Available: https://doi.org/10.1145/3465481.3465773

[5] L. Lerman and O. Markowitch, "Efficient profiled attacks on masking schemes," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 6, pp. 1445–1454, 2019. [Online]. Available: https://doi.org/10.1109/TIFS.2018.2879295

[6] S. Paguada, L. Batina, I. Buhan, and I. Armendariz, "Playing with blocks: Toward re-usable deep learning models for side-channel profiled attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2835–2847, 2022. [Online]. Available: https://doi.org/10.1109/TIFS.2022.3196273

[7] M. O. Choudary and M. G. Kuhn, "Efficient, portable template attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 2, pp. 490–501, 2018. [Online]. Available: https://doi.org/10.1109/TIFS.2017.2757440

[8] G. Hospodar, B. Gierlichs, E. D. Mulder, I. Verbauwhede, and J. Vandewalle, "Machine learning in side-channel analysis: a first study," *J. Cryptogr. Eng.*, vol. 1, no. 4, pp. 293–302, 2011. [Online]. Available: https://doi.org/10.1007/s13389-011-0023-x

[9] S. Picek, A. Heuser, A. Jovic, and L. Batina, "A systematic evaluation of profiling through focused feature selection," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 27, no. 12, pp. 2802–2815, 2019. [Online]. Available: https://doi.org/10.1109/TVLSI.2019.2937365

[10] A. Heuser, S. Picek, S. Guilley, and N. Mentens, "Lightweight ciphers and their side-channel resilience," *IEEE Trans. Computers*, vol. 69, no. 10, pp. 1434–1448, 2020. [Online]. Available: https://doi.org/10.1109/TC.2017.2757921

[11] H. Maghrebi, T. Portigliatti, and E. Prouff, "Breaking cryptographic implementations using deep learning techniques," in *Security, Privacy, and Applied Cryptography Engineering - 6th International Conference, SPACE 2016, Hyderabad, India, December 14-18, 2016, Proceedings*, ser. Lecture Notes in Computer Science, C. Carlet, M. A. Hasan, and V. Saraswat, Eds., vol. 10076. Springer, 2016, pp. 3–26. [Online]. Available: https://doi.org/10.1007/978-3-319-49445-6_1

[12] E. Prouff, R. Strullu, R. Benadjila, E. Cagli, and C. Dumas, "Study of deep learning techniques for side-channel analysis and introduction to ASCAD database," *IACR Cryptol. ePrint Arch.*, p. 53, 2018. [Online]. Available: http://eprint.iacr.org/2018/053

[13] L. Wouters, V. Arribas, B. Gierlichs, and B. Preneel, "Revisiting a methodology for efficient CNN architectures in profiling attacks," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 3, pp. 147–168, 2020. [Online]. Available: https://doi.org/10.13154/tches.v2020.i3.147-168

[14] P. Cao, C. Zhang, X. Lu, D. Gu, and S. Xu, "Improving deep learning based second-order side-channel analysis with bilinear CNN," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3863–3876, 2022. [Online]. Available: https://doi.org/10.1109/TIFS.2022.3216959

[15] S. Paguada, L. Batina, I. Buhan, and I. Armendariz, "Being patient and persistent: Optimizing an early stopping strategy for deep learning in profiled attacks," *IEEE Trans. Computers*, vol. 74, no. 3, pp. 875–886, 2025. [Online]. Available: https://doi.org/10.1109/TC.2023.3234205

[16] E. Cagli, C. Dumas, and E. Prouff, "Convolutional neural networks with data augmentation against jitter-based countermeasures - profiling attacks without pre-processing," in *Cryptographic Hardware and Embedded Systems - CHES 2017 - 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*, ser. Lecture Notes in Computer Science, W. Fischer and N. Homma, Eds., vol. 10529. Springer, 2017, pp. 45–68. [Online]. Available: https://doi.org/10.1007/978-3-319-66787-4_3

[17] G. Zaid, L. Bossuet, A. Habrard, and A. Venelli, "Methodology for efficient CNN architectures in profiling attacks," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 1, pp. 1–36, 2020. [Online]. Available: https://doi.org/10.13154/tches.v2020.i1.1-36

[18] L. Wu and S. Picek, "Remove some noise: On pre-processing of side-channel measurements with autoencoders," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 4, pp. 389–415, 2020. [Online]. Available: https://doi.org/10.13154/tches.v2020.i4.389-415

[19] U. Rioja, L. Batina, I. Armendariz, and J. L. Flores, "Towards human dependency elimination: AI approach to SCA robustness assessment," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3906–3921, 2022. [Online]. Available: https://doi.org/10.1109/TIFS.2022.3176189

[20] B. Gierlichs, L. Batina, P. Tuyls, and B. Preneel, "Mutual information analysis," in *Cryptographic Hardware and Embedded Systems - CHES 2008, 10th International Workshop, Washington, D.C., USA, August 10-13, 2008. Proceedings*, ser. Lecture Notes in Computer Science, E. Oswald and P. Rohatgi, Eds., vol. 5154. Springer, 2008, pp. 426–442. [Online]. Available: https://doi.org/10.1007/978-3-540-85053-3_27

[21] N. Bruneau, S. Guilley, A. Heuser, D. Marion, and O. Rioul, "Less is more - dimensionality reduction from a theoretical perspective," in *Cryptographic Hardware and Embedded Systems - CHES 2015 - 17th International Workshop, Saint-Malo, France, September 13-16, 2015, Proceedings*, ser. Lecture Notes in Computer Science, T. Güneysu and H. Handschuh, Eds., vol. 9293. Springer, 2015, pp. 22–41. [Online]. Available: https://doi.org/10.1007/978-3-662-48324-4_2

[22] J. Gao, X. Li, C. Ou, Z. Wang, and F. Yan, "Manifold learning side-channel attacks against masked cryptographic implementations," *IACR Cryptol. ePrint Arch.*, p. 1604, 2023. [Online]. Available: https://eprint.iacr.org/2023/1604

[23] Y. Won, X. Hou, D. Jap, J. Breier, and S. Bhasin, "Back to the basics: Seamless integration of side-channel pre-processing in deep neural networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 3215–3227, 2021. [Online]. Available: https://doi.org/10.1109/TIFS.2021.3076928

[24] T. Chen, S. Kornblith, M. Norouzi, and G. E. Hinton, "A simple framework for contrastive learning of visual representations," in *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 1597–1607. [Online]. Available: http://proceedings.mlr.press/v119/chen20j.html

[25] K. He, H. Fan, Y. Wu, S. Xie, and R. B. Girshick, "Momentum contrast for unsupervised visual representation learning," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*. Computer Vision Foundation / IEEE, 2020, pp. 9726–9735. [Online]. Available: https://doi.org/10.1109/CVPR42600.2020.00975

[26] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin, "Unsupervised feature learning via non-parametric instance discrimination," in *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 2018, pp. 3733–3742. [Online]. Available: http://openaccess.thecvf.com/content_cvpr_2018/html/Wu_Unsupervised_Feature_Learning_CVPR_2018_paper.html

[27] G. Perin, L. Wu, and S. Picek, "Exploring feature selection scenarios for deep learning-based side-channel analysis," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022, no. 4, pp. 828–861, 2022. [Online]. Available: https://doi.org/10.46586/tches.v2022.i4.828-861

[28] T. Schamberger, M. Egger, and L. Tebelmann, "Hide and seek: Using occlusion techniques for side-channel leakage attribution in cnns - an evaluation of the ASCAD databases," in *Applied Cryptography and Network Security Workshops - ACNS 2023 Satellite Workshops, ADSC, AIBlock, AIHWS, AIoTS, CIMSS, Cloud S&P, SCI, SecMT, SiMLA, Kyoto, Japan, June 19-22, 2023, Proceedings*, ser. Lecture Notes in Computer Science, J. Zhou, L. Batina, Z. Li, J. Lin, E. Losiouk, S. Majumdar, D. Mashima, W. Meng, S. Picek, M. A. Rahman, J. Shao, M. Shimaoka, E. O. Soremekun, C. Su, J. S. Teh, A. Udovenko, C. Wang, L. Y. Zhang, and Y. Zhauniarovich, Eds., vol. 13907. Springer, 2023, pp. 139–158. [Online]. Available: https://doi.org/10.1007/978-3-031-41181-6_8

[29] R. R. Selvaraju, A. Das, R. Vedantam, M. Cogswell, D. Parikh, and D. Batra, "Grad-cam: Why did you say that? visual explanations from deep networks via gradient-based localization," *CoRR*, vol. abs/1610.02391, 2016. [Online]. Available: http://arxiv.org/abs/1610.02391